

```
...
0 cat
1 dog
2 moose
```

The `range(len())` convention is straightforward but less than ideal because it can be difficult to read. Instead, pass the list or sequence to the built-in `enumerate()` function, which will return an integer for the index and the item at that index. For example, you can write the following pythonic code:

```
>>> # Pythonic Example
>>> animals = ['cat', 'dog', 'moose']
>>> for i, animal in enumerate(animals):
...     print(i, animal)
...
0 cat
1 dog
2 moose
```

The code you write will be slightly cleaner using `enumerate()` instead of `range(len())`. If you need only the items but not the indexes, you can still directly iterate over the list in a pythonic way:

```
>>> # Pythonic Example
>>> animals = ['cat', 'dog', 'moose']
>>> for animal in animals:
...     print(animal)
...
cat
dog
moose
```

Calling `enumerate()` and iterating over a sequence directly are preferable to using the old-fashioned `range(len())` convention.

Use the with Statement Instead of open() and close()

The `open()` function will return a file object that contains methods for reading or writing a file. When you're done, the file object's `close()` method makes the file available to other programs for reading and writing. You can use these functions individually. But doing so is unpythonic. For example,